

```

3  (* =====
4  UltrasphericalDerivativeZeros.wl
5
6  Certified rational enclosures for the zeros  $p'_{\{d,m,k\}}$  of the derivative
7  of the  $d$ -dimensional ultraspherical Bessel function of order  $m$ ,
8
9   $P_{\{d,m\}}(x) := x^{-(d/2-1)} \text{BesselJ}[m + d/2 - 1, x], \quad d \geq 3, m \geq 0,$ 
10
11  with the exceptional convention  $p'_{\{d,0,1\}} := 0$ .
12
13  This file implements the algorithm of
14
15  "Polya's conjecture for higher-dimensional Neumann balls",
16  N. Filonov, M. Levitin, I. Polterovich, D. A. Sher,
17
18  specifically the procedures of the Appendix "Rational enclosure for zeros
19  of derivatives of ultraspherical Bessel functions", used to fill the finite
20  gap in Section 5 ("Filling the gap"). All numbering below refers to that
21  paper: Definition (defn:ratn), Lemmas (lem:enclosures), (lem:trans),
22  (lem:pandj), the Lorch-Szego lower bound (eq:LS), and the sign conditions
23  (eq:signconditions).
24
25  NOTATION (as in the paper).
26   $nu = m + d/2 - 1$ 
27   $F_{nu}(t) = \text{Gamma}(nu+1) (x/2)^{-nu} J_{nu}(x) |_{\{t = x^2/4\}}$ 
28   $= \text{Sum}_j (-t)^j / (j! \text{Pi}_j(nu))$  (eq:deff)
29   $H_{\{d,m\}}(t) = m F_{\{m+d/2-1\}}(t) - (2t/(m+d/2)) F_{\{m+d/2\}}(t)$  (eq:defh)
30   $P'_{\{d,m\}}(x) = x^{-(d/2)} (x/2)^{(m+d/2-1)} / \text{Gamma}(m+d/2)$ 
31   $* H_{\{d,m\}}(x^2/4)$  (eq:uprime)
32  Both  $F_{nu}$  and  $H_{\{d,m\}}$  are entire with RATIONAL Taylor coefficients ( $nu$  is a
33  half-integer), and  $H_{\{d,m\}}(0) = m$ . In the notation of (defn:ratn),
34   $f_{\{nu,k\}} = j_{\{nu,k\}}^2/4$  are the positive zeros of  $F_{nu}$ , and
35   $h_{\{d,m,k\}} = (p'_{\{d,m,k\}})^2/4$  those of  $H_{\{d,m\}}$ .
36
37  MAIN TASK (Section 5).
38  ComputeListP[d, Lam, eps] implements the procedure Main(d, Lambda, eps).
39  d -- integer  $\geq 3$ ;
40  Lam -- rational (or integer)  $\Lambda > 0$ ;
41  eps -- rational (or integer) tolerance  $\epsilon > 0$ ;
42  Output: the complete list ListP_{\{d,Lam\}} of all pairs  $(m,k)$  with
43   $p'_{\{d,m,k\}} \leq \Lambda$ , returned as associations
44   $\langle | \text{"m"} \rightarrow m, \text{"k"} \rightarrow k, \text{"lower"} \rightarrow ., \text{"upper"} \rightarrow . \rangle$ 
45  where "upper" is the certified rational  $\overline{p'_{\{d,m,k\}}}$  with
46   $p'_{\{d,m,k\}} < \overline{p'_{\{d,m,k\}}} \leq \text{Min}[p'_{\{d,m,k\}} + \epsilon, \Lambda]$ ,
47  and "lower" is the companion  $\underline{p'_{\{d,m,k\}}} < p'_{\{d,m,k\}}$ , so that
48  the two-sided enclosure of width at most  $\epsilon$  mentioned in Section 5 is
49  available. For the exceptional zero the entry  $(0,1,0)$  is returned.

```

```

3      NeumannCountingFunction[listP, d] returns the exactly computed integer
4       $N^{\text{Neu}}_{\{B^d\}}(\text{Lam}) = \text{Sum of the multiplicities } \kappa_{\{d,m\}} \text{ over ListP,}$ 
5      the fourth column of Table (table:data).
6
7
8  VERIFICATION OF SECTION 5. Section 6 below assembles the re-indexed list
9  of Section 5 (the indices  $n_i$  and multiplicities  $\kappa_{\{d,m\}}$ ) and evaluates
10 the left-hand side of (eq:checkpolya) as an exact rational:
11      GapTable[] -- runs  $d = 3, \dots, 12$  at the tabulated
12                  thresholds  $\text{Lambda}^*_d$ , returning the
13                  data of Table (table:data);
14      PolyaCheckPassed[rows] -- True iff (eq:checkpolya) holds for all
15                               dimensions of the run;
16      GapTableForm[rows] -- the table in display layout (wrap in
17                           TeXForm for LaTeX).
18
19  DESIGN RULE. Every certificate rests exclusively on EXACT Integer/Rational
20 arithmetic, as required in Section 5: no N[...], no machine reals, and no
21 floating-point comparison occurs anywhere in Sections 1-5 below. Machine
22 precision appears ONLY in the clearly marked diagnostic Section 6, which
23 feeds into no certificate and exists purely as a cross-check.
24
25  VALIDATION. This file could not be executed in the environment where it
26 was written (no Wolfram kernel). Its logic was validated by running a
27 line-by-line mirror in exact rational arithmetic (Python/Fraction) against
28 independent high-precision evaluations of the zeros for
29  $(d, \text{Lam}) = (3,8), (4,10), (5,12), (6,15/2)$ : in each case the certified
30  $(m,k)$ -sets and the enclosures agreed exactly. Run DiagnosticCompare
31 (Section 6) as a first smoke test in an actual Wolfram session.
32
33 =====
34
35 (* =====
36 Section 0. Elementary exact helpers
37 =====
38
39 (*  $\text{nu} = m + d/2 - 1$ , and the quantity  $m(m+d-2)$  appearing in (eq:LS). With  $d$ 
40    an Integer,  $d/2$  is exact (Integer or Rational), so all of these are exact.
41    NuOf[d_Integer, m_Integer] := m + d/2 - 1;
42    EtaOf[d_Integer] := d/2 - 1;
43    KapOf[d_Integer, m_Integer] := m (m + d - 2);
44
45 (* Multiplicity  $\kappa_{\{d,m\}}$  of the spherical harmonics  $H_{\{d,m\}}$  on  $S^{(d-1)}$ ;
46    the single binomial formula is valid for all  $m \geq 0$  ( $\text{Binomial}[a,b] = 0$  for
47    integers  $0 \leq a < b$ ), e.g.  $d = 3$  gives  $2m + 1$ . Used to assemble the
48    indices  $n_i$  and the counting function of Section 5. *)
49 KappaMult[d_Integer, m_Integer] :=

```

```

Binomial[m + d - 1, d - 1] - Binomial[m + d - 3, d - 1];

(* A certified RATIONAL lower bound for Sqrt[q], q rational  $\geq 0$ , with dyadic
denominator  $2^{\text{prec}}$ . Floor of an exact algebraic number is exact in
Mathematica, so the result x0 satisfies  $x_0 \leq \text{Sqrt}[q]$  exactly. Used only
to enter the zero-free initial interval in BesselZeroBrackets below. *)
RatSqrtLower[q_ /; q  $\geq 0$ , prec_Integer: 16] :=
  Floor[2^prec Sqrt[q]]/2^prec;

(* Grid step of the march in BesselZeroBrackets. The paper fixes the value 3,
admissible because consecutive zeros satisfy  $j_{\nu,k} - j_{\nu,k-1} \geq \pi > 3$ 
for  $\nu \geq 1/2$  (Lorch-Szego, as quoted in the Certification clause of that
procedure). *)
$GridStep = 3;

(* Safety cap on the loops. The paper PROVES termination of every loop
(Certification/Termination clauses of the four procedures); the cap exists
only to turn a hypothetical implementation bug into a clean error message
rather than a hang. *)
$MaxIter = 100000;

BugAbort[tag_String] :=
  (Message[ComputeListP::bug, tag]; Abort[]);
ComputeListP::bug =
  "Internal safety cap reached in `1`; this contradicts the proved \
termination and indicates an implementation bug.";

(* Load-time collision guard. A definition whose name coincides with a
built-in (Protected) System symbol is silently rejected, after which every
call dispatches to the built-in -- a failure mode that is hard to diagnose
from the resulting messages. (This is exactly what happened when the
procedure below was named Refine, which is System`Refine.) The test uses
Names on strings, so it does not itself create any symbols. *)
83 With[{clash = Select[
84   {"NuOf", "EtaOf", "KapOf", "KappaMult", "RatSqrtLower", "BugAbort",
85     "FEnclosure", "HEnclosure", "NStart", "SignF", "SignH",
86     "BesselZeroBrackets", "RefineBracket", "ClassifyBracket",
87     "DerivativeWindows", "MCutoff", "ComputeListP",
88     "NeumannCountingFunction", "WeylConstantBound", "ExpandListP",
89     "PolyaRatio", "GapTable", "PolyaCheckPassed", "GapTableForm",
90     "PPPrimeFloat", "DiagnosticCompare"}],
91   Names["System`" <> #] != {} &]],
92   If[clash != {},
93     Print["FATAL: these names collide with built-in System symbols and \
94     must be renamed before use: ", clash];
95     Abort[]];
96
97

```

```

146 (* =====
147 Section 1. Certified enclosures of  $F_{\nu}(t)$  and  $H_{\{d,m\}}(t)$ 
148 (Lemma lem:enclosures)
149 =====
150
151 (* FEnclosure[ $\nu$ ,  $t$ ,  $n$ ] returns a rational interval  $\{lo, hi\}$  with
152  $lo \leq F_{\nu}(t) \leq hi$ ,
153 or $Failed if the hypothesis  $r_{\{\nu,N\}}(t) < 1$  of (lem:enclosures) is not yet
154 met (in which case the caller simply increases  $n$ ).
155
156 The partial sum  $F_{\{\nu,N\}}(t)$  is accumulated incrementally via the exact term
157 recursion  $a_0 = 1$ ,  $a_{j+1} = a_j (-t)/((j+1)(\nu+j+1))$ , after which
158  $\tau = |a_{\{N+1\}}| = t^{(N+1)} / ((N+1)! \text{Pi}_{\{N+1\}}(\nu))$ ,
159  $\rho = r_{\{\nu,N\}}(t) = t / ((N+2)(\nu+N+2))$ ,
160 and the radius is  $\delta_{\{F_{\{\nu,N\}}\}}(t) = \tau/(1 - \rho)$ , exactly as in
161 (lem:enclosures). Every quantity is Rational. *)
162 FEnclosure[ $\nu$ _,  $t$ _,  $n\_Integer$ ] := Module[{term = 1, s = 1, j, tau, rho, r},
163 Do[
164 term = term * (-t)/((j + 1) (nu + j + 1));
165 s = s + term,
166 {j, 0, n - 1}]; (* now s =  $F_{\{\nu,n\}}$ , term =  $a_n$  *)
167 tau = Abs[term] * t/((n + 1) (nu + n + 1)); (* =  $|a_{\{n+1\}}|$  *)
168 rho = t/((n + 2) (nu + n + 2)); (* =  $r_{\{\nu,n\}}(t)$  *)
169 If[rho ≥ 1, Return[$Failed]];
170 r = tau/(1 - rho); (* =  $\delta_{\{F_{\{\nu,n\}}\}}$  *)
171 {s - r, s + r}];
172
173 (* HEnclosure[ $d$ ,  $m$ ,  $t$ ,  $n$ ]: rational interval containing  $H_{\{d,m\}}(t)$ , obtained
174 from the two F-enclosures by exact interval arithmetic. Since  $t \geq 0$  and
175 the coefficients  $m$  and  $2t/(\nu+1) = 2t/(m+d/2)$  are  $\geq 0$ , midpoints and radii
176 combine linearly, giving exactly the  $H_{\{d,m,N\}}$  and  $\delta_{\{H_{\{d,m,N\}}\}}$  of
177 (lem:enclosures). *)
178 HEnclosure[ $d\_Integer$ ,  $m\_Integer$ ,  $t$ _,  $n\_Integer$ ] :=
179 Module[{ $\nu$  = NuOf[ $d$ ,  $m$ ], e1, e2, c, mid, rad},
180 e1 = FEnclosure[ $\nu$ ,  $t$ ,  $n$ ];
181 e2 = FEnclosure[ $\nu + 1$ ,  $t$ ,  $n$ ];
182 If[e1 === $Failed || e2 === $Failed, Return[$Failed]];
183 c = 2 t/(nu + 1); (* =  $2t/(m + d/2)$  *)
184 mid = m (e1[[1]] + e1[[2]])/2 - c (e2[[1]] + e2[[2]])/2;
185 rad = m (e1[[2]] - e1[[1]])/2 + c (e2[[2]] - e2[[1]])/2;
186 {mid - rad, mid + rad}];
187
188 (* =====
189 Section 2. Procedure Sign( $X$ ,  $x$ )
190 =====
191
192
193 (* Starting truncation order. The paper's workflow sets  $N = 8$ ; any starting

```

value is admissible, and $n \sim 2 \sqrt{t}$ merely starts past the hump of the series so that the radius already decays fast. Ceiling of an exact $\sqrt{t}$ is exact, so this stays within rational arithmetic. *)

```
NStart[t_] := Max[8, Ceiling[2 Sqrt[t]]];
```

```
(* SignF[nu, x]      = certified Sign of F_nu(x^2/4)      in {+1, -1};
   SignH[d, m, x]    = certified Sign of H_{d,m}(x^2/4)    in {+1, -1}.
```

These are the two instantiations X in $\{F_{\text{nu}}, H_{\{d,m\}}\}$ of the procedure $\text{Sign}(X, x)$. Both take the point in the variable x (rational, ≥ 0) and query the series at $t = x^2/4$, again rational. At $x = 0$ the exact values $F_{\text{nu}}(0) = 1$ and $H_{\{d,m\}}(0) = m$ are used instead of an enclosure; the latter is the exception noted in the Input clause of the procedure Refine .

Doubling loop as in the workflow: compute the enclosure; if it excludes 0, return the common sign of its endpoints; otherwise double N .

Certification: by (lem:enclosures).

Termination: by (lem:trans), all zeros of F_{nu} and $H_{\{d,m\}}$ are irrational, so the true value $X(x)$ is nonzero at rational x and the loop must exit. *)

```
SignF[nu_, x_] := Module[{t, n, enc, res},
```

```
  If[x == 0, Return[1]];                                (* F_nu(0) = 1      *)
```

```
  t = x^2/4; n = NStart[t];
```

```
  res = Catch[
```

```
    Do[
```

```
      enc = FEnclosure[nu, t, n];
```

```
      If[enc != $Failed && (enc[[1]] > 0 || enc[[2]] < 0),
```

```
        Throw[Sign[enc[[1]]]]];
```

```
      n = 2 n,
```

```
      {$MaxIter}];
```

```
    $Failed];
```

```
  If[res === $Failed, BugAbort["SignF"]];
```

```
  res];
```

```
SignH[d_Integer, m_Integer, x_] := Module[{t, n, enc, res},
```

```
  If[x == 0, Return[Sign[m]]];                            (* H_{d,m}(0) = m  *)
```

```
  t = x^2/4; n = NStart[t];
```

```
  res = Catch[
```

```
    Do[
```

```
      enc = HEnclosure[d, m, t, n];
```

```
      If[enc != $Failed && (enc[[1]] > 0 || enc[[2]] < 0),
```

```
        Throw[Sign[enc[[1]]]]];
```

```
      n = 2 n,
```

```
      {$MaxIter}];
```

```
    $Failed];
```

```
  If[res === $Failed, BugAbort["SignH"]];
```

```
  res];
```

189

190

191

192

```

242 (* =====
243 Section 3. Procedure BesselZeroBrackets(nu, Lambda)
244 =====
245
246 (* Returns the list
247   { {a_1, b_1, sa_1, sb_1}, ..., {a_K, b_K, sa_K, sb_K} }
248   of rational brackets  $j_{\{nu,k\}}$  in  $(a_k, b_k)$ ,  $b_k - a_k = 3$ ,  $a_K \geq \text{Lam}$ ,
249   together with the certified signs  $sa_k, sb_k$  of  $F_{nu}$  at the two endpoints
250   (cached for the later bisections).
251
252   Certification (as in the paper): consecutive zeros satisfy
253    $j_{\{nu,k\}} - j_{\{nu,k-1\}} \geq \text{Pi} > 3$  for  $nu \geq 1/2$  (Lorch-Szego), so each
254   interval of length 3 contains at most one zero, which is simple; the march
255   starts inside the zero-free initial interval, since  $j_{\{nu,1\}} > nu$ . Rational
256   grid points are never zeros by (lem:trans), so all signs are genuine.
257   Hence the recorded brackets enumerate, in increasing order and without
258   omission, the zeros  $j_{\{nu,1\}}, j_{\{nu,2\}}, \dots$ .
259   Termination: there are finitely many zeros in  $(0, \text{Lam}]$ .
260
261   NOTE ON THE ENTRY POINT. The paper's workflow starts at  $x_0 = nu$ . The
262   code starts instead at a rational lower bound for  $\text{Sqrt}[nu^2 - 1/4] \leq nu$ ,
263   which lies in the same zero-free interval and is therefore equally valid,
264   merely taking a few more grid steps. To match the paper's workflow exactly
265   replace the line defining  $x_0$  by  $x_0 = nu$ ; ( $nu$  is a half-integer, hence
266   exactly rational, so this stays within exact arithmetic). *)
267 BesselZeroBrackets[nu_, Lam_] := Module[
268   {x0, xi, si, xn, sn, brackets = {}, res},
269   x0 = If[nu == 1/2, 0, RatSqrtLower[nu^2 - 1/4]];
270   xi = x0; si = SignF[nu, xi]; (* = +1, as  $j_{\{nu,1\}} > nu \geq x_0$  *)
271   res = Catch[
272     Do[
273       xn = xi + $GridStep;
274       sn = SignF[nu, xn];
275       If[si sn < 0, (* certified sign change: *)
276         AppendTo[brackets, {xi, xn, si, sn}]; (* exactly one zero here *)
277         If[xi ≥ Lam, Throw[brackets]]; (* stop:  $a_K \geq \text{Lambda}$  *)
278         xi = xn; si = sn,
279       {$MaxIter}]];
280   $Failed];
281   If[res === $Failed, BugAbort["BesselZeroBrackets"]];
282   res];

```

```

285 (* =====
286 Section 4. Procedure Refine(X, (a,b)) [code name: RefineBracket]
287 =====
288
289 (* One bisection step on a bracket {a, b, sa, sb} carrying a certified sign
290 change of the function whose sign oracle is signFun (a pure function
291  $x \mapsto \pm 1$ ); at a zero left endpoint the exact value  $H_{\{d,m\}}(0) = m$  is used
292 in place of Sign(X, 0), as stated in the Input clause of the procedure.
293 The rational midpoint is never a zero by (lem:trans), so the returned
294 half-width bracket again carries a certified sign change and contains the
295 same unique zero.
296
297 NAME. The paper calls this procedure Refine. That name CANNOT be used
298 here: Refine is a built-in Wolfram symbol (Refine[expr, assumptions]) and
299 is Protected, so a definition of it is silently rejected and every call
300 dispatches to the built-in. We therefore use RefineBracket. *)
301 RefineBracket[signFun_, {l_, r_, sl_, sr_}] := Module[{mid, sm},
302   mid = (l + r)/2; (* dyadic-friendly exact midpoint *)
303   sm = signFun[mid];
304   If[sm == sl, {mid, r, sm, sr}, {l, mid, sl, sm}]];
305
306
307 (* =====
308 Section 5. Procedure Main(d, Lambda, eps)
309 =====
310
311 (* ClassifyBracket implements the two exit conditions (i) / (ii) shared by
312 step 1 and step 2.3 of Main. Given a bracket certified to contain a unique
313 zero z of the function with sign oracle signFun, it bisects until either
314 (i)  $b \leq \text{Lam}$  and  $b - a \leq \text{eps}$ : return {"include", a, b}, the final
315 two-sided enclosure  $a < z < b \leq \text{Min}[\text{Lam}, z + \text{eps}]$ ; or
316 (ii)  $a \geq \text{Lam}$ : return {"exclude"}.
317 Exactly one exit is reached after finitely many steps: by (lem:trans),
318  $z \neq \text{Lam}$ , so  $z < \text{Lam}$  forces (i) and  $z > \text{Lam}$  forces (ii). *)
319 ClassifyBracket[signFun_, br0_List, Lam_, eps_] := Module[{br = br0, res},
320   res = Catch[
321     Do[
322       Which[
323         br[[2]] ≤ Lam && br[[2]] - br[[1]] ≤ eps,
324         Throw[{"include", br[[1]], br[[2]]}],
325         br[[1]] ≥ Lam,
326         Throw[{"exclude"}],
327         True,
328         br = RefineBracket[signFun, br]],
329     {$MaxIter}]];
330   $Failed];
331   If[res === $Failed, BugAbort["ClassifyBracket"]];
332   res];

```

(* DerivativeWindows implements steps 2.1 and 2.2 of Main for $m \geq 1$.

Step 2.1: refine each bracket (a_k, b_k) of BesselZeroBrackets, bisecting on F_{nu} , until the ultraspherical derivative sign conditions

$\text{Sign}(H_{\{d,m\}}, a_k) = \text{Sign}(H_{\{d,m\}}, b_k) = (-1)^k$ (eq:signcond-
hold. Step termination: as a_k increases to $j_{\{nu,k\}}$ and b_k decreases to $j_{\{nu,k\}}$, both signs eventually equal $(-1)^k$ by (lem:pandj), which gives
 $\text{sign } P'_{\{d,m\}}(j_{\{nu,k\}}) = (-1)^k$; each RefineBracket halves the width. Ref-
preserves $a_K \geq \text{Lam}$, since a_K can only increase.

Step 2.2: return the ultraspherical derivative windows

$W_0 = (0, a_1)$, $W_k = (b_k, a_{\{k+1\}})$, $k = 1, \dots, K-1$,
each as a bracket-with-signs for the $H_{\{d,m\}}$ oracle; at the left endpoint of
 W_0 the exact value $H_{\{d,m\}}(0) = m > 0$ is used. By (lem:pandj) and
(eq:signconditions), W_k contains exactly one zero of $H_{\{d,m\}}$, namely
 $p'_{\{d,m,k+1\}}$; no zero of $H_{\{d,m\}}$ in $(0, j_{\{nu,K\}})$ lies outside the closure
of the union of the W_k , while $p'_{\{d,m,K+1\}} > j_{\{nu,K\}} > a_K \geq \text{Lam}$.

Subtle case handled automatically: consecutive zeros $j_{\{nu,k\}}$ may fall in
ADJACENT grid cells (their gap can be as small as $\text{Pi} > 3$), so that initially
 $b_k = a_{\{k+1\}}$. The sign conditions at that shared point would be
contradictory $((-1)^k \text{ versus } (-1)^{(k+1)})$, so the conditioning loop
necessarily keeps refining until b_k and $a_{\{k+1\}}$ separate; the certified
signs then force $b_k < p'_{\{d,m,k+1\}} < a_{\{k+1\}}$, a nonempty window. *)

```

307 DerivativeWindows[d_Integer, m_Integer, dirBrs_List] := Module[
308   {nu = NuOf[d, m], K = Length[dirBrs], brs = dirBrs,
309    target, k, windows, done},
310   (* -- Step 2.1: enforce the sign conditions (eq:signconditions) ----- *)
311   Do[
312     target = (-1)^k;
313     done = Catch[
314       Do[
315         If[SignH[d, m, brs[[k, 1]]] == target &&
316            SignH[d, m, brs[[k, 2]]] == target,
317            Throw[True]];
318         brs[[k]] = RefineBracket[SignF[nu, #] &, brs[[k]],
319           {$MaxIter}];
320       False];
321     If[done == False, BugAbort["DerivativeWindows"]],
322     {k, K}];
323   (* -- Step 2.2: assemble the windows W_0, ..., W_{K-1} ----- *)
324   windows = Table[
325     If[k == 0,
326       {0, brs[[1, 1]], 1, -1},
327       {brs[[k, 2]], brs[[k + 1, 1]], (-1)^k, (-1)^(k + 1)}],
328     {k, 0, K - 1}];

```



```

windows];

(* The cut-off  $M_{\{d,\text{Lambda}\}} = \text{Max}\{ m \geq 0 : m(m + d - 2) < \text{Lambda}^2 \}$  quoted in
Main, justified by the Lorch-Szego lower bound (eq:LS),
 $p'_{\{d,m,1\}} > \text{Sqrt}(m(m+d-2))$  for  $m \geq 1$ . Exact integer march. *)
MCutoff[d_Integer, Lam_] := Module[{M = 0},
  While[(M + 1) (M + d - 1) < Lam^2, M++]; M];

(* ----- *)
(* ComputeListP[d, Lam, eps] -- the procedure Main; see the header for the *)
(* output contract (the list ListP of Section 5). *)
(* ----- *)
ComputeListP[d_Integer /; d ≥ 3,
  Lam_ /; Element[Lam, Rationals] && Lam > 0,
  eps_ /; Element[eps, Rationals] && eps > 0] :=
Module[{L = {}, nu, brs, wins, res, M, m, k},

  (* ---- Step 0: initialise with the exceptional zero  $p'_{\{d,0,1\}} = 0$  ---- *)
  AppendTo[L, <|"m" → 0, "k" → 1, "lower" → 0, "upper" → 0|>];

  (* ---- Step 1: the case  $m = 0$  -----
By (lem:pandj),  $p'_{\{d,0,k\}} = j_{\{d/2,k-1\}}$ , so the zeros are classified
directly on the brackets for  $\text{nu} = d/2$  using the F-oracle. They increase
in k, so we may stop at the first exclusion; by construction  $a_K \geq \text{Lam}$ ,
so exit (ii) occurs at the latest at  $k = K$ , and all zeros with  $k > K$ 
satisfy  $j_{\{d/2,k\}} > j_{\{d/2,K\}} > a_K \geq \text{Lam}$ . ----- *)
  nu = d/2;
  brs = BesselZeroBrackets[nu, Lam];
  Do[
    res = ClassifyBracket[SignF[nu, #] &, brs[[k]], Lam, eps];
    If[res[[1]] == "include",
      AppendTo[L, <|"m" → 0, "k" → k + 1,
        "lower" → res[[2]], "upper" → res[[3]]|>],
      Break[]],
    {k, Length[brs]}}];

  (* ---- Step 2: the cases  $1 \leq m \leq M_{\{d,\text{Lambda}\}}$  ----- *)
  M = MCutoff[d, Lam];
  Do[
    nu = NuOf[d, m];
    brs = BesselZeroBrackets[nu, Lam]; (* exhaustive  $j_{\{\text{nu},k\}}$ ,  $k \leq K$  *)
    wins = DerivativeWindows[d, m, brs]; (*  $W_0, \dots, W_{\{K-1\}}$  *)
    Do[
      (* window  $W_{\{k-1\}}$  contains exactly one zero of  $H_{\{d,m\}}$ :  $p'_{\{d,m,k\}}$  *)
      res = ClassifyBracket[SignH[d, m, #] &, wins[[k]], Lam, eps];
      If[res[[1]] == "include",
        AppendTo[L, <|"m" → m, "k" → k,
```

```

                                "lower" → res[[2]], "upper" → res[[3]]|>],
      Break[]],                                (* zeros increase in k *)
      {k, Length[wins]}},
      {m, 1, M}];

(* ---- Step 3: return ListP ----- *)
SortBy[L, {#["m"] &, #["k"] &}]];

(* Exact value of the Neumann counting function of the unit ball at Lambda,
   the fourth column of Table (table:data):
      N^Neu_{B^d}(Lam) = Sum over (m,k) in ListP of kappa_{d,m}. *)
NeumannCountingFunction[result_List, d_Integer] :=
  Total[KappaMult[d, #["m"]] & /@ result];

```

```

442 (* =====
443 Section 6. Assembling the table of Section 5 and checking Polya's
444 conjecture, i.e. the inequality (eq:checkpolya)
445 =====
446
447 (* A rational UPPER bound for the Weyl constant
448
449     w_d = 2^(-d) / Gamma(1 + d/2)^2
450
451     (the constant of Weyl's law for the unit ball, since |B^d| = omega_d and
452     w_d = omega_d^2/(2 Pi)^d). For even d this is already rational. For odd d
453     a factor 1/Pi remains, and, exactly as stated in Section 5, we replace 1/Pi
454     by 1/3; since Pi > 3 this OVERSTATES w_d, so the resulting ratio overstates
455     the left-hand side of (eq:checkpolya) and the check stays rigorous.
456
457     Implementation note: multiplying by Pi clears the factor for odd d, and the
458     result is then asserted to be rational before it is returned -- so a silent
459     failure to reduce cannot slip through into a "certified" number. *)
460 WeylConstantBound[d_Integer] := Module[{w, r},
461   w = 2^-d/Gamma[1 + d/2]^2;
462   r = If[OddQ[d], (w Pi)/3, w];          (* w Pi is rational for odd d *)
463   If[! (Head[r] === Integer || Head[r] === Rational),
464     Print["FATAL: WeylConstantBound[" , d, "] did not reduce to a rational: " ,
465       r]; Abort[]];
466   r];
467
468 (* ExpandListP re-arranges and re-indexes ListP exactly as in Section 5:
469 the entries are sorted by their certified upper bounds and given the
470 cumulative indices
471     n_1 = 1,      n_{i+1} = n_i + kappa_{d,m_i},
472 so that the Neumann eigenvalues of B^d satisfy
473     mu_{n_i} = ... = mu_{n_i + kappa_{d,m_i} - 1} < (upper_i)^2.
474 The exceptional entry (0,1,0) has upper bound 0 and therefore sorts first,

```

giving $(n_1, m_1, k_1) = (1, 0, 1)$ as required.

Why sorting by the UPPER bounds is legitimate even though the enclosures may in principle overlap: every entry placed before position i has upper bound $\leq \text{upper}_i$, hence its true zero is $< \text{upper}_i$. So the $n_i - 1$ eigenvalues accumulated before position i are all $< (\text{upper}_i)^2$ whatever the true order of two nearly equal zeros happens to be, and the displayed inequality for $\mu_{\{n_i\}}$ holds regardless. Ties are broken by (m,k) purely for reproducibility.

```

The exact rationals are retained in "lower"/"upper"; "lower(N)"/"upper(N)"
are decimal approximations for display only and enter no certificate. *)
ExpandListP[d_Integer, listP_List, digits_Integer : 6] :=
Module[{s, out, n = 1, kap, e, j},
442   s = SortBy[listP, {#["upper"] &, #["m"] &, #["k"] &}];
443   out = ConstantArray[Null, Length[s]];
444   Do[
445     e = s[[j]];
446     kap = KappaMult[d, e["m"]];
447     out[[j]] = <|"n" → n, "m" → e["m"], "k" → e["k"],
448               "multiplicity" → kap,
449               "lower" → e["lower"], "upper" → e["upper"],
450               "lower(N)" → N[e["lower"], digits],
451               "upper(N)" → N[e["upper"], digits]|>;
452     n = n + kap,
453     {j, Length[s]}}];
454   out];
455
456 (* PolyRatio returns the left-hand side of (eq:checkpolya),
457
458     max_{i ≥ 2} w_d (upper_i)^d / (n_i - 1) ,
459
460 as an EXACT rational number (all inputs are rational, w_d being the bound
461 above). The first entry, with n_1 = 1, is excluded: it would divide by
462 n_1 - 1 = 0. Polya's conjecture holds on the range covered by ListP
463 precisely when this value is < 1. *)
464 PolyRatio[d_Integer, expanded_List] :=
465 Module[{w = WeylConstantBound[d], sel},
466   sel = Select[expanded, #["n"] > 1 &];
467   If[sel === {},
468     Print["FATAL: PolyRatio[" , d, "]: no entries with n > 1."]; Abort[]];
469   Max[w #["upper"]^d/(#["n"] - 1) & /@ sel];
470
471 (* The thresholds Lambda*_d = Ceiling[(tau*_d)^3 d^(3/2)] of Section 5,
472 as tabulated in (table:data), for d = 3, ..., 12. *)
473 $LambdaStar = <|3 → 19, 4 → 22, 5 → 25, 6 → 29, 7 → 34,
474               8 → 39, 9 → 44, 10 → 49, 11 → 54, 12 → 60|>;

```

```

(* GapTable runs the whole verification of Section 5 for the given dimensions
   and returns one association per dimension, holding the exact Polya ratio
   together with the data of (table:data). The full expanded list is kept
   under the key "expanded" for inspection.
   Everything except the "... (N) display columns is exact. *)
GapTable[eps_ : 1/100, ds_ : Range[3, 12], verbose_ : True] :=
Module[{rows = {}, listP, exp, ratio, Lam, d, cnt},
  Do[
    Lam = $LambdaStar[d];
    listP = ComputeListP[d, Lam, eps];
    exp = ExpandListP[d, listP];
    ratio = PolyaRatio[d, exp];
    cnt = NeumannCountingFunction[listP, d];
442   If[verbose,
443     Print["d = ", d, ", Lambda* = ", Lam, ", K_d = ", Length[listP],
444           ", N^Neu = ", cnt, ", Polya ratio = ", N[ratio, 6],
445           ", < 1: ", ratio < 1]];
446   AppendTo[rows,
447     <|"d" → d, "Lambda*" → Lam, "K" → Length[listP], "NNeu" → cnt,
448       "ratio" → ratio, "ratio(N)" → N[ratio, 4], "expanded" → exp|>,
449     {d, ds}];
450   rows];
451
452 (* The certified conclusion: True exactly when (eq:checkpolya) holds in every
453    dimension of the run. The comparison is between exact rationals. *)
454 PolyaCheckPassed[rows_List] := AllTrue[rows, #["ratio"] < 1 &];
455
456 (* Display in the layout of (table:data); wrap in TeXForm for the LaTeX
457    source of the table (note: TeXForm, not TextForm). *)
458 GapTableForm[rows_List] :=
459   TableForm[
460     {#["d"], #["Lambda*"], #["K"], #["NNeu"], #["ratio(N)"]} & /@ rows,
461     TableHeadings → {None,
462       {"d", "\Lambda*_d", "K_d", "N^Neu", "approx. LHS"}}];
463
464 (* =====
465    Section 7. DIAGNOSTICS ONLY -- machine-precision cross-checks.
466    Nothing in this section is used by, or feeds into, any certificate above.
467    =====
468
469 (* Float value of p'_{d,m,k}, NON-RIGOROUS, for cross-checking only.
470
471    For m = 0 the identity p'_{d,0,k} = j_{d/2,k-1} of (lem:pandj) is used. For
472    m ≥ 1 we bisect g(x) = m J_nu(x) - x J_{nu+1}(x) -- the bracketed factor
473    of P'_{d,m} in the paper's displayed formula for the derivative -- on the
474    interlacing bracket (j_{nu,k-1}, j_{nu,k}) of (eq:pvsj), whose endpoint

```

signs are $(-1)^{(k-1)}$ and $(-1)^k$ by (lem:pandj). We START from the known left-end sign and never evaluate g at the endpoints themselves (near $x = 0$ the value $m J_{nu}(10^{-6})$ underflows at machine precision). Evaluation is at 30 significant digits; 60 bisection steps on an interval of length $< \sim 2 \pi$ give $\sim 15+$ digits, ample for a plausibility check. *)

```

PPrimeFloat[d_Integer, m_Integer, k_Integer] := Module[
  {nu, g, lo, hi, signLo, mid, wp = 30},
  If[m == 0,
    If[k == 1, Return[0],
      Return[N[BesselJZero[d/2, k - 1], wp]]];
  nu = NuOf[d, m]; (* exact half-integer *)
  g[x_] := m BesselJ[nu, x] - x BesselJ[nu + 1, x];
  lo = If[k == 1, N[10^-6, wp],
    N[BesselJZero[nu, k - 1], wp] + 10^-20];
  hi = N[BesselJZero[nu, k], wp] - 10^-20;
  signLo = (-1)^(k - 1); (* sign of g just right of j_{nu,k-1} *)
  Do[
    mid = (lo + hi)/2;
    If[Sign[g[mid]] == signLo, lo = mid, hi = mid],
    {60}];
  (lo + hi)/2];

```

(* DiagnosticCompare[d, Lam, eps]: runs the certified algorithm, then checks (with floats!) that each returned enclosure indeed straddles the float zero. A non-numeric float value is itself reported as a failure of the DIAGNOSTIC (never of the certificate). Smoke test for the IMPLEMENTATION, not part of any proof. *)

```

DiagnosticCompare[d_Integer, Lam_, eps_] := Module[
  {L, ok = True, z},
  L = ComputeListP[d, Lam, eps];
  Scan[
    (z = PPrimeFloat[d, #["m"], #["k"]];
    Which[
      !NumericQ[z],
        ok = False;
        Print["DIAGNOSTIC FAILURE at (m,k) = ", {#["m"], #["k"]},
          ": float solver returned ", z],
      !(N[#["lower"], 30] ≤ z ≤ N[#["upper"], 30]),
        ok = False;
        Print["MISMATCH at (m,k) = ", {#["m"], #["k"]},
          ": float ", N[z], " not in ",
          N[{#["lower"], #["upper"]}]] &, L];
  Print[Length[L], " zeros certified <= ", N[Lam],
    "; N^Neu = ", NeumannCountingFunction[L, d],
    "; enclosures consistent with floats: ", ok];
  {ok, L}];

```

559

560

561

562

563

564

565

566

567

568

569

```

(* Example usage (uncomment to run):

(* -- a single dimension, reproducing one row of Table (table:data): *)

listP = ComputeListP[3, 19, 1/100];          (* d = 3, Lambda^*_3 = 19 *)
Length[listP]                                (* K_3 = 50 *)
NeumannCountingFunction[listP, 3]            (* N^Neu = 570 *)
Dataset[ExpandListP[3, listP]]               (* the expanded list *)
PolyaRatio[3, ExpandListP[3, listP]]        (* exact; approx 0.9061 *)

(* -- the full verification of Section 5, d = 3, ..., 12: *)

rows = GapTable[];                          (* prints progress per d *)
PolyaCheckPassed[rows]                      (* must be True *)
GapTableForm[rows]
TeXForm[GapTableForm[rows]]                (* LaTeX source *)

(* -- optional float cross-check of the enclosures in one dimension: *)

DiagnosticCompare[5, 25, 1/100];

*)

```